

# NEXORA

*Learn. Connect. Grow.*

---

## Day 2 — Design Documentation

---

System Architecture, Class Design, ER Design & API Specification for the Mentor Slot Booking MVP

# 1. Project Overview

## Project Description

Nexora — “Learn. Connect. Grow.”

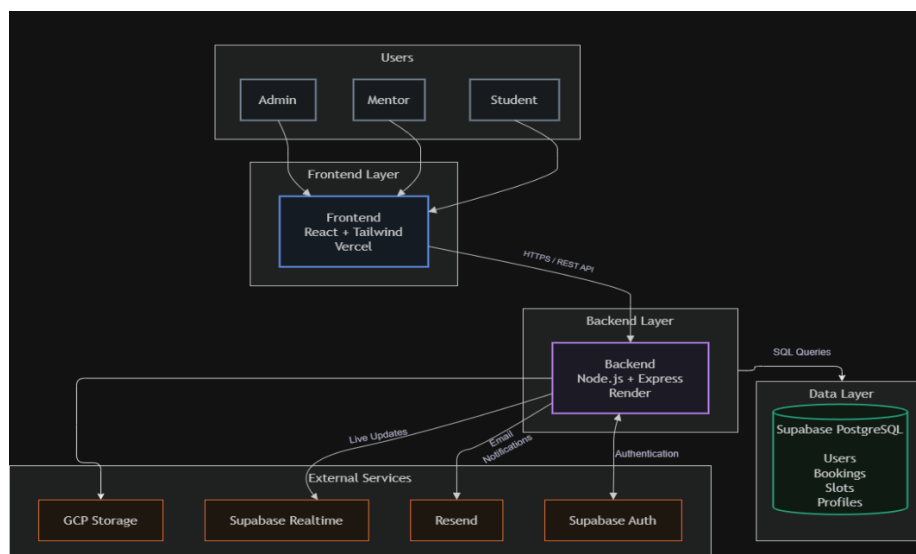
Nexora is a mentorship and career-growth platform built to connect students and trainees with mentors through a simple, structured booking system. The current build is the MVP: a One-on-One Mentor Slot Booking System, where mentors publish their available time slots and students book a session directly, with automatic confirmation and reminder emails.

The platform is designed so that this booking system is the foundation of a larger product. Future modules — internship tracking, assignment management, event registration, analytics, community, and AI-based recommendations — are planned to build on top of the same user accounts and database structure introduced in this MVP.

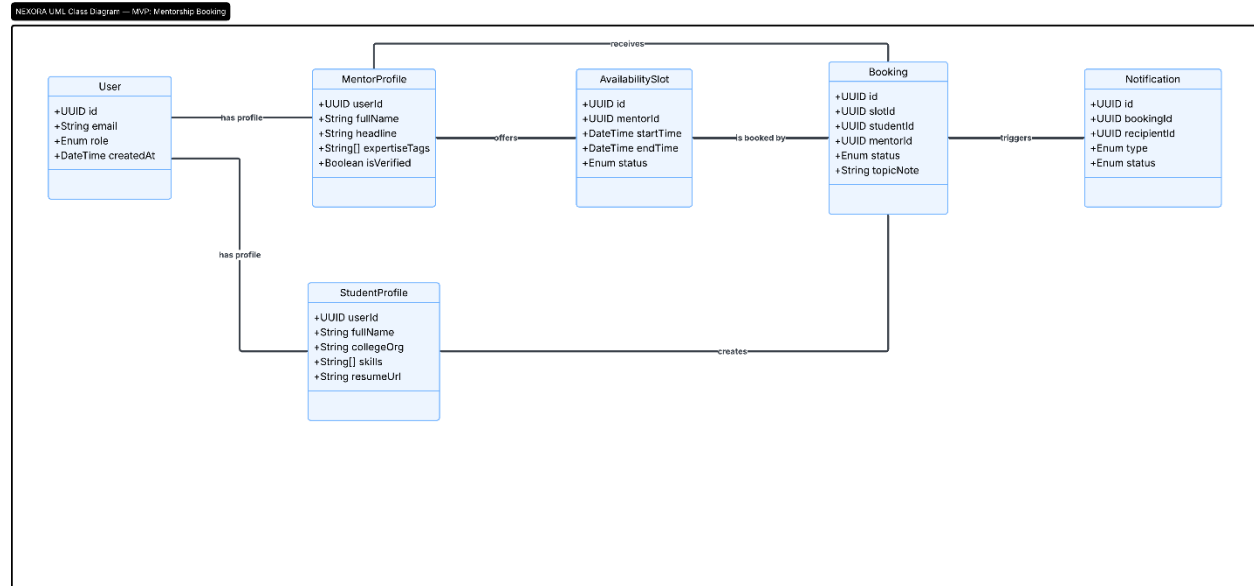
## Objective

- Allow students to discover mentors and book one-on-one sessions without manual back-and-forth scheduling.
- Allow mentors to publish their availability and manage incoming bookings from one dashboard.
- Send automatic confirmation and reminder notifications so sessions are not missed.
- Build the system on a clean, simple architecture that can later support the full Nexora platform without major rework.

# 2. Architecture Diagram



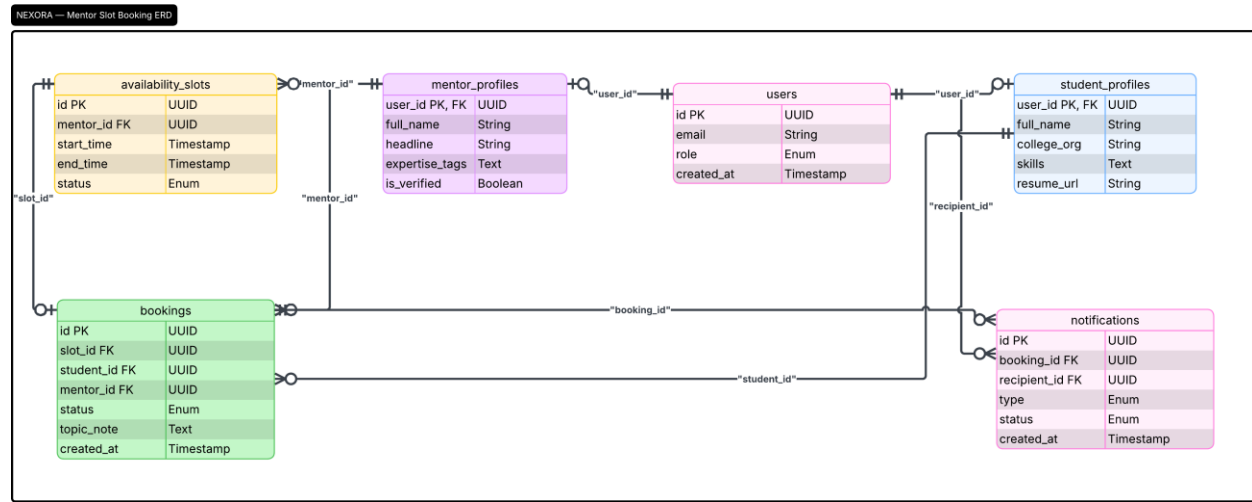
### 3. Class Diagram



#### Relationships and Cardinalities

| From             | To               | Relationship                                         | Cardinality |
|------------------|------------------|------------------------------------------------------|-------------|
| User             | StudentProfile   | One user has one student profile (if role = student) | 1 : 1       |
| User             | MentorProfile    | One user has one mentor profile (if role = mentor)   | 1 : 1       |
| MentorProfile    | AvailabilitySlot | One mentor publishes many slots                      | 1 : N       |
| AvailabilitySlot | Booking          | One slot becomes at most one booking                 | 1 : 0..1    |
| StudentProfile   | Booking          | One student can make many bookings                   | 1 : N       |
| Booking          | Notification     | One booking generates many notifications             | 1 : N       |

## 4. ER Diagram



### Table Relationships

| Relationship                          | Cardinality | Enforced By                  |
|---------------------------------------|-------------|------------------------------|
| users to student_profiles             | 1 : 1       | Foreign key on user_id       |
| users to mentor_profiles              | 1 : 1       | Foreign key on user_id       |
| mentor_profiles to availability_slots | 1 : N       | Foreign key mentor_id        |
| availability_slots to bookings        | 1 : 0..1    | Unique constraint on slot_id |
| student_profiles to bookings          | 1 : N       | Foreign key student_id       |
| bookings to notifications             | 1 : N       | Foreign key booking_id       |

## 5. API List

All endpoints are prefixed with /api. Endpoints other than authentication and the public mentor directory require the user to be logged in.

### 5.1 Authentication APIs

| Endpoint           | Method | Description                                                  |
|--------------------|--------|--------------------------------------------------------------|
| /api/auth/register | POST   | Create a new account through Supabase Auth and assign a role |
| /api/auth/login    | POST   | Log in and receive a session                                 |
| /api/auth/logout   | POST   | Log out and end the current session                          |

## 5.2 User APIs

| Endpoint             | Method | Description                         |
|----------------------|--------|-------------------------------------|
| /api/users/me        | GET    | Get the logged-in user's profile    |
| /api/users/me        | PATCH  | Update the logged-in user's profile |
| /api/users/me/avatar | POST   | Upload or replace a profile picture |

## 5.3 Mentor APIs

| Endpoint                  | Method | Description                                     |
|---------------------------|--------|-------------------------------------------------|
| /api/mentors              | GET    | Search and filter the mentor directory by skill |
| /api/mentors/:id          | GET    | Get one mentor's public profile                 |
| /api/mentors/:id/slots    | GET    | Get a mentor's open availability slots          |
| /api/mentors/me/slots     | POST   | Publish a new availability slot (mentor only)   |
| /api/mentors/me/slots/:id | DELETE | Remove an unbooked slot (mentor only)           |

## 5.4 Booking APIs

| Endpoint                   | Method | Description                               |
|----------------------------|--------|-------------------------------------------|
| /api/bookings              | POST   | Book an open slot                         |
| /api/bookings/me           | GET    | List the logged-in user's bookings        |
| /api/bookings/:id          | GET    | Get details of one booking                |
| /api/bookings/:id/cancel   | PATCH  | Cancel a booking and release the slot     |
| /api/bookings/:id/complete | PATCH  | Mark a session as completed (mentor only) |

## 5.5 Admin APIs

| Endpoint                      | Method | Description                                   |
|-------------------------------|--------|-----------------------------------------------|
| /api/admin/mentors/pending    | GET    | List mentor profiles waiting for verification |
| /api/admin/mentors/:id/verify | PATCH  | Approve a mentor's profile                    |
| /api/admin/bookings           | GET    | View all bookings on the platform             |

## 6. Future Scope

---

### **If given 6 months and a budget, how would this project expand?**

With 6 months and a small dedicated budget, the next step would not be to rebuild Nexora — it would be to add new modules on top of the same users table, the same authentication system, and the same notification pipeline already built for the MVP. Each module below is additive, not a redesign.

#### ***Internship Tracker***

A new section where students can log internship applications and track their stage (applied, interview, offer, rejected). It reuses the existing student profile and login system, just adding a new applications table linked to the same user.

#### ***Assignment Manager***

Lets mentors assign small tasks to their mentees after a session and review submissions. It builds on the existing booking relationship between a mentor and a student, so assignments are naturally tied to an existing mentorship connection.

#### ***Event Registration***

Allows organizations to publish workshops or webinars that students can register for, using the same slot and booking pattern already proven in the MVP, just applied to group events instead of one-on-one sessions.

#### ***AI Mentor Recommendation***

Once enough booking and feedback data exists, a recommendation feature can suggest the best-matching mentors to a student based on their skills and past session history, instead of the student searching manually.

#### ***Mobile Application***

A React Native version of the same platform, reusing the existing backend APIs without any backend changes — the mobile app would simply be a new client talking to the same Express API.

#### ***Analytics Dashboard***

An expanded reporting view for admins and organizations showing booking trends, completion rates, and mentor activity over time, built from data that the MVP is already collecting in the bookings and notifications tables.

#### ***Community Forum***

A discussion space where students and mentors can post questions and share resources, scoped by topic or program, using the same authentication and role system already in place.

**Why this approach works:** Every future module connects back to the same users table and reuses the same login system and notification pipeline introduced in the MVP. This means Nexora can grow feature by feature without needing to redesign its foundation.